

What Does an Operation Cost?

Lecture Notes — Week 2: Running Time, Asymptotic Notation, and Best/Worst/Average Case

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 21, 2026

2.1 Why We Need a Cost Model

Read alongside: Sedgewick & Wayne, *Algorithms*, §1.4, p. 172 — the opening motivating question [4]; Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 1 (chapter-level) [1]; Neapolitan & Naimipour, *Foundations of Algorithms*, Ch. 1–2 (chapter-level) [3].

Week 1 built the vocabulary of contracts: an ADT states what a structure can do, and a data structure is one way of honouring that contract. What that chapter left completely unanswered is a second, equally basic question: given two implementations that both honour the same contract, which one should we actually use? This chapter answers that question, in three parts. First, how do we measure the work an operation does? Second, how do we compare two correct solutions without running both of them on every possible input? Third, how do we say how much memory a solution needs? The plan for getting there is short: first count the work exactly, then compress the count into a simple class, then apply the vocabulary to two concrete searches.

2.1.1 The running example: a symbol table that must stay fast

Return to the expression processor introduced in Week 1. It reads an expression such as $(a+b)*c$ and stores each identifier — a , b , c — in a symbol table that must later be looked up by name. A single line of input has only two or three identifiers, but a real calculator session run over hours may hold thousands. Week 1 left the symbol table’s implementation deliberately open: “a list for now, a hash table later.” This week’s question sharpens that promise into something we can actually reason about: as the table grows, how do we predict whether a lookup stays fast — *before* we build either implementation? The answer cannot be “we will time it and see, once it’s built.” What is needed is a model that predicts the answer in advance.

Consider two ways of finding the value bound to a name. The first scans the table from the top, name by name, until it finds a match or runs out of entries. The second keeps the table sorted and repeatedly jumps to the middle of the remaining range, discarding half of it at each step. Both are correct: each returns the same value for a name that is present, and each correctly reports “not found” for a name that is absent. With ten names, both finish before a human would notice any difference. With ten million names, the first approach is thousands of times slower than the second — even though neither algorithm’s logic has changed by a single line. This is the central distinction the chapter is built on: **correctness and cost are two different questions**. Week 1 answered the first question — does the structure do what it promises? This week answers the second — what does honouring that promise *cost*?

2.1.2 “It felt fast” is not an argument

Sedgewick & Wayne open their own analysis chapter with exactly this tension: “*How long will my program take? Why does my program run out of memory?*” — and they immediately note that these are questions “much too vague to be answered precisely” without a mathematical model [4] (p. 172). Three observations justify insisting on a model rather than a stopwatch. Timing one run on one machine measures that run on that machine, not the algorithm in any general sense. A faster machine shifts the absolute numbers but tells the same story about which algorithm is better, or worse. What is actually needed is a way to count the *work* an algorithm does as a function of the *size* of its input, independent of clock speed, compiler, or the specific machine running it.

Definition (Cost model). *Count the basic operations an algorithm performs as a function of n , the size of the input; then compress that count into one asymptotic class [1, 3].*

The cost model has three moving parts, and the rest of this chapter works through them in order. **Measure:** pick one basic operation and count how often it runs. **Compress:** once the count is known, keep only the term that grows fastest as n grows, and discard everything else. **State the case:** report whether the count describes the best, worst, or average input of size n , since — as Section 2.3 makes precise — the same algorithm can cost very different amounts on different inputs of the same size. Horowitz & Sahni (Ch. 1) and Neapolitan & Naimipour (Ch. 1–2) both treat algorithm analysis this way, at the chapter level [1, 3]. To compare two solutions at all, we first have to count the work each one does — which is exactly where Section 2.2 begins.

2.2 Counting the Work: The Cost Function

Read alongside: Sedgewick & Wayne, *Algorithms*, §1.4 (general treatment) [4].

Definition (Basic operation). *A single step whose cost is roughly constant, regardless of the surrounding input: a comparison, an assignment, an array access, or one arithmetic operation [4].*

The cost model does not count microseconds; it counts steps. Machine time depends on clock speed, cache behaviour, and compiler optimizations — none of which is a property of the algorithm itself. What *is* a property of the algorithm is how many times it performs some chosen operation, and the model’s whole trick is choosing an operation that *dominates* the algorithm’s running time. For a search, that operation is the comparison `A[i] == key`: every other step (incrementing a loop counter, returning a value) happens at most as often as the comparison does, so counting the comparison alone gives the right *shape* of the growth, even though it is a deliberate simplification. Exact machine time is unknowable in general; the step count is not.

Definition (Cost function $T(n)$). *$T(n)$ is the number of basic operations an algorithm performs on an input of size n .*

Two conventions attach to $T(n)$ for the rest of the course. First, n always counts the elements under discussion — here, the length of the array `A[0..n-1]`, continuing Week 1’s notation exactly. Second, $T(n)$ is a *function*, not a single number: it answers “how many steps?” for every possible input size at once, not just for one run. Everything about n and arrays is inherited from Week 1; $T(n)$ itself is the new idea this week introduces.

Example 2.2.1 (Linear search: counting the comparisons). *The first algorithm this chapter analyses scans an array from the front:*

```
int linear_search(int A[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (A[i] == key)
```

```

        return i;
    return -1;
}

```

The basic operation is the comparison $A[i] == \text{key}$: every pass through the loop body performs exactly one of these comparisons, so counting comparisons counts loop iterations. The loop's index i runs from 0 up to $n-1$, so the maximum possible number of comparisons is n . The open question — and the whole point of the rest of this section — is how many of those up-to- n comparisons actually happen on a given run, because that number depends on where key sits in the array, or whether it sits anywhere in the array at all.

Example 2.2.2 ($T(n)$ depends on the input, not just on n). Take the array $A = [10, 20, 30, 40, 50]$, so $n = 5$, and run `linear_search` twice: once for $\text{key} = 10$ and once for $\text{key} = 99$. Trace each run comparison by comparison.

For $\text{key} = 10$: the very first comparison, $A[0] == \text{key}$, i.e. $10 = 10$, succeeds immediately, and the function returns 0. This run performs exactly 1 comparison.

For $\text{key} = 99$: compare $A[0] == 99$ ($10 \neq 99$, comparison 1), then $A[1] == 99$ ($20 \neq 99$, comparison 2), then $A[2] == 99$ ($30 \neq 99$, comparison 3), then $A[3] == 99$ ($40 \neq 99$, comparison 4), then $A[4] == 99$ ($50 \neq 99$, comparison 5). The loop's index now reaches $n = 5$, the loop condition $i < n$ fails, and the function returns -1. This run performs exactly $5 = n$ comparisons.

Both runs search an array of the same size, $n = 5$, yet one costs 1 step and the other costs 5. The cost of `linear_search` is therefore not a single number attached to n ; it depends on which input of size n is given. This is exactly why the cost model must describe cost over kinds of input rather than over n alone — the subject of Section 2.3.

2.3 Best, Worst, and Average Case

Read alongside: Karumanchi, *Data Structures and Algorithms Made Easy*, Ch. 1, pp. 16–61 (PDF, chapter-level) [2].

Definition (Case (best/worst/average)). Three questions asked of the same algorithm on inputs of size n [2]: the **best case** is the minimum cost over all inputs of size n ; the **worst case** is the maximum cost over all inputs of size n ; the **average case** is the expected cost, given some assumed distribution over inputs of size n .

A *case* is a property of the *input*; a *bound* such as O or Θ (Section 2.4) is a property of the *function* $T(n)$ itself. The two ideas are orthogonal, and the discipline this course insists on — stated once here and enforced for the rest of the term — is to pair them, never to confuse them. “ $O(n)$ ” on its own is an incomplete claim; “ $O(n)$ in the worst case” is a complete one.

Example 2.3.1 (Linear search: all three cases). Return to `linear_search` on an array $A[0..n-1]$, still counting the comparison $A[i] == \text{key}$.

Best case. $\text{key} == A[0]$: the very first comparison succeeds, exactly as in the $\text{key} = 10$ run above. This costs 1 comparison, independent of n .

Worst case. key is absent from the array, or sits at $A[n-1]$, the last position: every one of the n positions must be compared before the loop can conclude (either by exhausting the loop, or by matching on the final comparison). This costs n comparisons.

Average case. Assume a successful search in which key is equally likely to be found at any of the n positions, each with probability $1/n$. If key sits at position k (using 1-based counting of

positions for this sum, $k = 1, \dots, n$), the search performs exactly k comparisons before finding it. The expected number of comparisons is therefore

$$\frac{1}{n}(1 + 2 + \dots + n).$$

The sum $1 + 2 + \dots + n$ has the standard closed form $\frac{n(n+1)}{2}$ (pair the first and last terms, the second and second-last, and so on; each pair sums to $n+1$, and there are $n/2$ such pairs). Substituting this closed form into the expectation gives

$$\frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}.$$

So the average-case cost of a successful linear search is $\frac{n+1}{2}$ comparisons — roughly half the array, which matches the intuition that a successful search on average stops about halfway through.

2.3.1 Which input forces the worst case?

A natural check on the worst-case claim above: for a 5-element array, which input actually forces `linear_search` to perform all 5 comparisons? Three candidates are worth comparing. `key` equal to `A[0]` costs only 1 comparison — this is the *best* case, not the worst. `key` equal to `A[4]`, the last element, does cost 5 comparisons, since every earlier position must be checked and rejected before the loop reaches index 4. `key` equal to a value nowhere in the array also costs 5 comparisons, since the loop must exhaust every index without ever finding a match. Both of the last two candidates cost the same $n = 5$ comparisons for this specific array, so both are legitimate witnesses of the worst case on this one input — but they are not equally informative as a *general* argument. “Key at the last position” is a fact about one particular array; a different array could put its key anywhere, and the argument would need to be rebuilt each time. “Key absent from the array” forces all n comparisons *regardless of what the array’s contents are or how it is arranged*, which is exactly the property a worst-case bound needs to hold for *every* input of size n , not merely for one cleverly chosen example. This is why textbook worst-case arguments for linear search are usually phrased in terms of an absent key: it is the input class that makes the bound a genuine guarantee rather than an anecdote.

2.4 Asymptotic Notation: Big-O, Big-Omega, and Big-Theta

Read alongside: Sedgewick & Wayne, *Algorithms*, §1.4, p. 187 — the order-of-growth classification table [4].

Exact counts such as $\frac{n+1}{2}$ or $\frac{n(n-1)}{2}$ are precise but unwieldy, and much of their precision is not actually useful for comparing algorithms. What we want instead is the *shape* of the growth — which is exactly what asymptotic notation captures.

2.4.1 Why ignore the constants?

Suppose $T(n) = 3n + 2$, and the input size doubles. The doubling argument: however large the constant 3 is, the cost still roughly doubles when n does, because the dominant term $3n$ scales linearly with n while the constant $+2$ becomes negligible for large n . The functions $3n + 2$ and $7n - 5$ both *grow like* n in this sense: they differ only by a constant factor, not by their fundamental

shape. Constant factors matter enormously to a machine actually running the code — a program that is 7 times slower than another, for the same input, is a real difference a user will notice — but constant factors do not matter to *how the cost scales* as n grows, which is the question asymptotic analysis is built to answer. Lower-order terms, such as the $+2$ in $3n+2$, disappear entirely once n is large enough that the dominant term swamps them. **Asymptotic analysis keeps the dominant term and drops the constants.**

2.4.2 Orders of growth

A handful of growth classes cover almost every algorithm this course will meet, and Table 1 lists them in increasing order, following Sedgewick & Wayne’s own classification (p. 187) [4].

Order	Name	Typical of
1	constant	one statement
$\log n$	logarithmic	binary search
n	linear	linear search
$n \log n$	linearithmic	mergesort
n^2	quadratic	nested loops
n^3	cubic	triple loops

Table 1: Orders of growth, from cheapest to most expensive — cf. Sedgewick & Wayne, *Algorithms*, p. 187 [4].

Figure 2.1 draws the same four of these classes side by side over a small range of n , making visible just how differently they climb: $\log n$ is nearly flat, n is a straight line, $n \log n$ curves gently above the line, and n^2 pulls sharply away from all three.

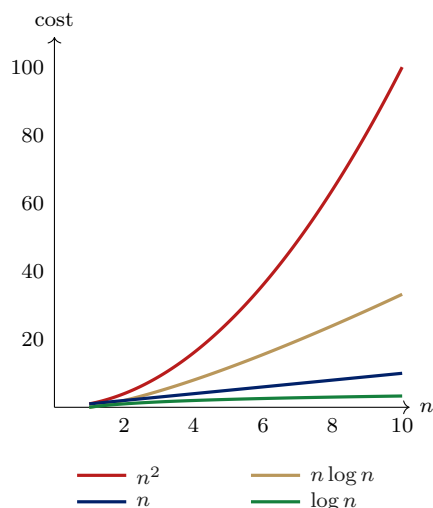


Figure 2.1: Growth of $\log_2 n$, n , $n \log_2 n$, and n^2 over $n = 1$ to 10 — cf. Sedgewick & Wayne, *Algorithms*, p. 187 [4].

2.4.3 Big-O: an upper bound

Definition (Big-O). $T(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that

$$T(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0.$$

In words: $T(n)$ grows no faster than $g(n)$, up to a constant factor and after some starting point n_0 . O answers the question “at worst, roughly how much work?” — it is a one-sided guarantee, and possibly a loose one, since any function that grows even more slowly than $g(n)$ also satisfies the same $O(g(n))$ bound.

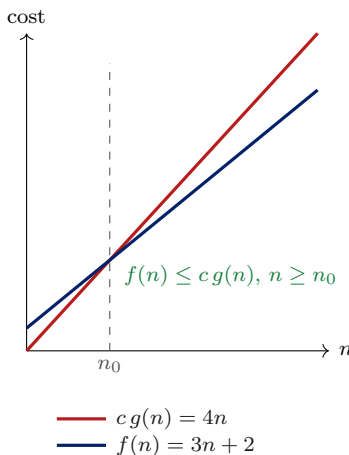


Figure 2.2: Big- O as an upper bound: $f(n) = 3n + 2$ is eventually at or below $c g(n) = 4n$, for every $n \geq n_0$.

Example 2.4.1 ($T(n) = 3n + 2$ is $O(n)$). *Claim: choosing $c = 4$ and $n_0 = 2$ satisfies the definition. Check the boundary first, at $n = n_0 - 1 = 1$: $3(1) + 2 = 5$, while $4 \cdot 1 = 4$, and $5 > 4$ — the inequality fails at $n = 1$, which is exactly why n_0 cannot be taken as 0 or 1 here. Now check $n = n_0 = 2$: $3(2) + 2 = 8$, while $4 \cdot 2 = 8$, and $8 \leq 8$ — the inequality holds, with equality. For any $n > 2$, the linear term $3n$ grows at the same rate as $4n$ but $4n$ started with a $1n$ head start built into the constant 4 versus 3, so once the $+2$ has been absorbed at $n = 2$ the gap only widens: $4n - (3n + 2) = n - 2 \geq 0$ for all $n \geq 2$. So $3n + 2 \leq 4n$ holds for every $n \geq 2$, confirming $c = 4$, $n_0 = 2$ is a valid choice, and therefore $T(n) = O(n)$. The exact function was $3n + 2$; the asymptotic class the analysis reports is simply $O(n)$ — reading it aloud, “the running time is of order n ,” already throws away the 3 and the 2 as constants that do not change the shape of the growth.*

2.4.4 Big-Omega: a lower bound

Definition (Big-Omega). $T(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that

$$T(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0.$$

In words: $T(n)$ grows *at least* as fast as $g(n)$, up to a constant factor. Where O answers “at worst, how much work,” Ω answers “at best, how much work” — a guarantee on how *little* the cost can be, not how much.

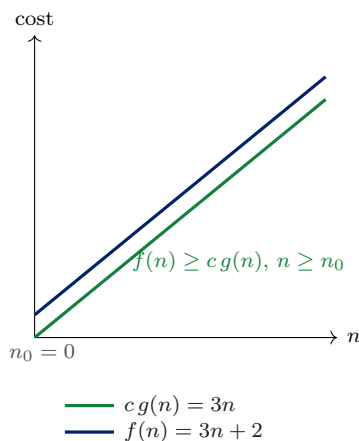


Figure 2.3: Big- Ω as a lower bound: $f(n) = 3n + 2$ is always at or above $c g(n) = 3n$.

Example 2.4.2 ($T(n) = 3n + 2$ is $\Omega(n)$). *Claim: choosing $c = 3$ and $n_0 = 0$ satisfies the definition. The inequality to check is $3n + 2 \geq 3n$ for every $n \geq 0$; subtracting $3n$ from both sides gives $2 \geq 0$, which is true unconditionally, for every value of n . So the inequality holds for all $n \geq 0$, and in particular for all $n \geq n_0 = 0$, confirming $T(n) = \Omega(n)$ with $c = 3$.*

2.4.5 Big-Theta: a tight bound

Definition (Big-Theta). $T(n) = \Theta(g(n))$ when $T(n)$ is both $O(g(n))$ and $\Omega(g(n))$ — that is, when $g(n)$ sandwiches $T(n)$ from above and below, each up to its own constant.

In words: $T(n)$ grows *exactly* like $g(n)$, up to a constant factor, in both directions at once. Θ is the strongest of the three claims, and should be used only when both bounds genuinely meet.

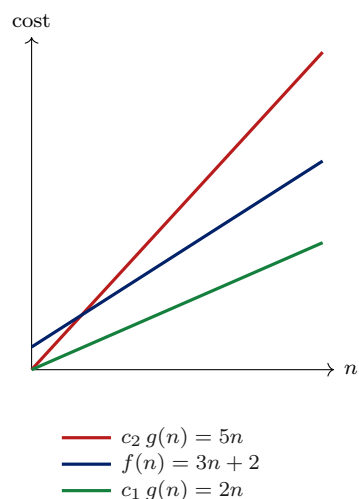


Figure 2.4: Big- Θ as a tight bound: $f(n) = 3n + 2$ is sandwiched between $c_1 g(n) = 2n$ below and $c_2 g(n) = 5n$ above.

Example 2.4.3 ($T(n) = 3n + 2$ is $\Theta(n)$). *This is where the O and Ω results above are combined. Lower bound. Take $c_1 = 2$: the inequality $3n + 2 \geq 2n$ simplifies (subtract $2n$ from both sides) to*

$n + 2 \geq 0$, which holds for every $n \geq 0$. **Upper bound.** Take $c_2 = 5$: the inequality $3n + 2 \leq 5n$ simplifies (subtract $3n$, then divide by 2) to $1 \leq n$, so this bound holds for $n \geq 1$. Combining the two: for every $n \geq 1$, both $2n \leq 3n + 2$ and $3n + 2 \leq 5n$ hold simultaneously, i.e. $2n \leq 3n + 2 \leq 5n$ — exactly the sandwich Θ requires, with $c_1 = 2$, $c_2 = 5$, $n_0 = 1$. So $T(n) = 3n + 2$ is $\Theta(n)$: it is bounded below by $2n$ and above by $5n$.

2.4.6 The strict siblings: little-o and little-omega

Definition (Little-o and little-omega). $T(n) = o(g(n))$ means $T(n)/g(n) \rightarrow 0$ as $n \rightarrow \infty$ — a strict upper bound. $T(n) = \omega(g(n))$ means $T(n)/g(n) \rightarrow \infty$ as $n \rightarrow \infty$ — a strict lower bound.

The strict forms tighten O and Ω by ruling out the boundary case where the two functions actually grow at the *same* rate. For example, $n = o(n^2)$, since $n/n^2 = 1/n \rightarrow 0$; but $2n$ is *not* $o(n)$, since $2n/n = 2$, a nonzero constant, not a ratio tending to 0. Likewise $n^2 = \omega(n)$, since $n^2/n = n \rightarrow \infty$; but $2n$ is *not* $\omega(n)$, since $2n/n = 2$ again, not a ratio tending to infinity. These strict forms are rarely needed in this course; the vast majority of claims made from here on will use O , Ω , or Θ .

2.4.7 Reading notation correctly

A bound must always name its case, or the claim is incomplete. “Linear search is $O(n)$ ” on its own does not say *in which case* — best, worst, or average — the bound applies, and different cases of the same algorithm can have entirely different bounds. Correctly stated claims name both the bound and the case: **linear search is $O(n)$ in the worst case**, and **linear search is $\Omega(1)$ in the best case**. Two common errors to avoid: “**binary search is $O(\log n)$** ” states no case at all, leaving the reader to guess which input the bound describes; and “ **$T(n) = O(n)$, so exactly n steps**” confuses an asymptotic *class* with an exact *count* — $O(n)$ describes a whole family of functions that grow no faster than some constant multiple of n , not the single function “ n .” **Pair every complexity claim with the case, and name the operation being counted.**

2.5 Two Searches, Two Costs

With the vocabulary of Sections 2.2–2.4 in hand, this section applies it to the two searches introduced informally in Section 2.1: the linear scan and the sorted-array jump.

2.5.1 Linear search, analysed

Counting the comparison `A[i] == key` as before, the three cases established in Section 2.3 translate directly into asymptotic classes. The worst case costs n comparisons, i.e. $O(n)$ — exhibited by the trace where `key` is absent and every `A[i]` is examined exactly once. The best case costs 1 comparison — exhibited by `key == A[0]`. The average case costs $\frac{n+1}{2}$ comparisons, which is $\Theta(n)$, since $\frac{n+1}{2}$ is sandwiched between $c_1 n$ and $c_2 n$ for suitable constants (indeed $\frac{n+1}{2} \geq \frac{1}{2}n$ for all $n \geq 0$, and $\frac{n+1}{2} \leq n$ for all $n \geq 1$). Every one of these three claims names its case explicitly, and each is matched to the specific input that exhibits it — exactly the discipline Section 2.4 insisted on.

2.5.2 Binary search needs a sorted table

Binary search is only *correct* when the array is sorted; this is a precondition of the algorithm, an invariant that must hold before a single comparison is made — not an option to be checked

later. The reason is structural: the sorted order is precisely what makes halving valid. After one comparison against the middle element, binary search discards an entire half of the remaining range, trusting that every element in the discarded half is on the wrong side of **key**. That trust is only justified when the array is sorted; on an unsorted array, the middle element’s relationship to its neighbours says nothing about where a target value might sit, so discarding a whole half can silently discard the very half containing **key** — the algorithm would then report “not found” for a key that is actually present, without any error or warning. Keeping a table sorted is itself a cost, paid on every insertion, in exchange for the lookup speed binary search buys once the table is sorted — a trade-off this chapter’s vocabulary will let us state precisely in Section 2.5.4. The anti-pattern to guard against, stated once and worth repeating: never apply binary search without first stating and checking the sorted precondition.

Example 2.5.1 (Binary search).

```
int binary_search(int A[], int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (A[mid] == key) return mid;
        else if (A[mid] < key) lo = mid + 1;
        else
            hi = mid - 1;
    }
    return -1;
}
```

Each pass compares $A[mid]$ against **key** and shrinks the active range $[lo, hi]$ by roughly half: if $A[mid]$ is too small, the entire lower half (including mid) is discarded by setting $lo = mid + 1$; if $A[mid]$ is too large, the entire upper half is discarded by setting $hi = mid - 1$. The array A must already be sorted in ascending order for this discarding logic to be valid, per the precondition above.

Figure 2.5 shows the array A used in the worked trace below, with lo , mid , and hi marking the state after the loop’s first pass computes $mid = lo + (hi - lo) / 2$, landing on index 4 (value 30).

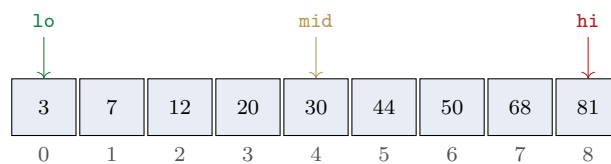


Figure 2.5: Binary search on $A = [3, 7, 12, 20, 30, 44, 50, 68, 81]$ ($n = 9$): the first pass’s lo , mid , hi pointers. Since $30 < 68$, the left half (indices 0–4) is discarded and lo jumps to 5.

Example 2.5.2 (Binary search trace, **key** = 68). Searching the array of Figure 2.5 for **key** = 68, tracing lo , hi , mid , and the comparison at each pass, one pass at a time:

Pass 1. $lo = 0$, $hi = 8$. Compute $mid = 0 + (8 - 0) / 2 = 4$. Compare $A[4] = 30$ against **key** = 68: since $30 < 68$, the target must be in the upper half, so set $lo = mid + 1 = 5$.

Pass 2. $lo = 5$, $hi = 8$. Compute $mid = 5 + (8 - 5) / 2 = 5 + 1 = 6$. Compare $A[6] = 50$ against **key** = 68: since $50 < 68$, set $lo = mid + 1 = 7$.

Pass 3. $lo = 7$, $hi = 8$. Compute $mid = 7 + (8 - 7) / 2 = 7 + 0 = 7$ (integer division truncates). Compare $A[7] = 68$ against **key** = 68: they are equal, so the function returns $mid = 7$ immediately.

<i>pass</i>	<i>lo</i>	<i>hi</i>	<i>mid</i>	<i>A[mid]</i>
1	0	8	4	30 < 68
2	5	8	6	50 < 68
3	7	8	7	68 = 68 — found

Three passes locate **key** = 68 among $n = 9$ values. For comparison, a linear scan starting from index 0 would have to check several of the array's early cells before ever reaching index 7 where 68 sits; in the worst case a linear scan over this same array checks every one of its 9 cells. Each binary-search pass discards half of the remaining interval, and that halving is the entire engine behind the logarithmic bound derived next.

2.5.3 Binary search, analysed

Each pass halves the active interval [*lo*, *hi*]: after k passes, at most $\lceil n/2^k \rceil$ elements remain under consideration (this is an upper bound on the number of surviving elements, consistent with — though not identical to — the specific counts in the trace above, since the trace is one concrete run and the formula bounds every possible run). The loop stops as soon as the interval becomes empty, i.e. once $\lceil n/2^k \rceil$ reaches 1 or below, which happens once k is about $\log_2 n$. Concretely, for $n = 9$: after 1 pass at most $\lceil 9/2 \rceil = 5$ elements remain, after 2 passes at most $\lceil 9/4 \rceil = 3$ remain, after 3 passes at most $\lceil 9/8 \rceil = 2$ remain, and after 4 passes at most $\lceil 9/16 \rceil = 1$ remains — consistent with the worked trace needing only 3 passes to finish (the specific target's position let it finish slightly before the general bound is exhausted). **Binary search is $O(\log n)$ in the worst case** — log here is the base-2 logarithm, but the base is itself only a constant factor, which asymptotic notation drops regardless of which base is chosen. **Binary search is $\Omega(1)$ in the best case**, exhibited when **key** sits exactly at **A[mid]** on the very first pass.

Example 2.5.3 (A nested loop: checking for duplicates). *A third algorithm worth analysing checks whether the symbol table already holds the same name twice:*

```
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        if (A[i] == A[j])
            return 1; /* duplicate found */
```

For a fixed value of the outer index i , the inner loop's index j runs from $i + 1$ up to $n - 1$, which is $n - (i + 1) = n - i - 1$ iterations, each performing one comparison $A[i] == A[j]$. Summing over every value of i from 0 to $n - 2$ (the last value for which the inner loop can run at all) gives the total comparison count

$$T(n) = \sum_{i=0}^{n-2} (n - i - 1).$$

Substitute $j = n - i - 1$: as i ranges over $0, 1, \dots, n - 2$, the new index j ranges over $n - 1, n - 2, \dots, 1$ — the same set of values, counted in the opposite order. So the sum is exactly

$$\sum_{j=1}^{n-1} j = \frac{(n-1)n}{2} = \frac{n(n-1)}{2},$$

using the same closed form for a sum of consecutive integers derived in Section 2.3. Expanding, $\frac{n(n-1)}{2} = \frac{n^2 - n}{2}$, whose dominant term is $\frac{1}{2}n^2$ — a constant multiple of n^2 , so $T(n) = \Theta(n^2)$: since

the loop as written always runs to completion unless a duplicate is actually encountered and triggers the early **return 1**, the full double loop — and hence the $\Theta(n^2)$ count — describes every run in which no duplicate exists to trigger an early exit.

n	$\log_2 n$	n	$n \log_2 n$	n^2
10	≈ 3	10	≈ 33	10^2
10^3	≈ 10	10^3	$\approx 10^4$	10^6
10^6	≈ 20	10^6	$\approx 2 \cdot 10^7$	10^{12}

Table 2: The orders of growth from Table 1, evaluated at concrete input sizes.

Table 2 makes the shape of these growth rates concrete: as n grows from 10 to a million, $\log_2 n$ barely moves — from about 3 to about 20 — while n^2 explodes from 100 to a trillion. This is the practical difference between a search that remains usable as the data grows and one that does not.

2.5.4 The symbol table payoff

Returning to the running example one more time: the calculator’s identifier lookup, analysed as n grows, now has a precise trade-off attached to each of its three possible homes. As an **unsorted list** (the implementation Week 6 will build), lookup costs $O(n)$ in the worst case — the whole table may need scanning. As a **sorted array**, lookup costs $O(\log n)$ in the worst case via binary search — but keeping the array sorted costs something on every insertion, since a new identifier may need to be slotted into the middle of an already-sorted range. As a **hash table** (the implementation Week 10 will build), lookup costs $O(1)$ *expected* — the entire reason a hash table will eventually be worth building. This chapter supplies the vocabulary that makes this three-way trade-off precise; Weeks 6 and 10 will act on it by actually building the list and the hash table and confirming the bounds empirically.

2.6 Space Complexity

Definition (Space complexity $S(n)$). $S(n)$ is the amount of memory an algorithm uses beyond its input — its auxiliary space — as a function of n .

Two components make up an algorithm’s total memory use: **input space**, the n elements of the input itself, counted separately since every algorithm operating on an array of size n trivially needs at least that much memory to hold the array; and **auxiliary space**, the *extra* working memory the algorithm needs beyond the input — extra local variables, temporary arrays, or recursion stack frames. $S(n)$ is stated in exactly the same asymptotic classes as $T(n)$: $O(1)$, $O(\log n)$, $O(n)$, and so on. The two are separated because “this algorithm uses n memory cells” is an uninteresting observation when the input already occupies n cells regardless of the algorithm; the auxiliary space is the number that actually distinguishes one algorithm’s memory footprint from another’s.

Both of this chapter’s search algorithms keep only a small, fixed handful of local variables, independent of n . **linear_search** uses one index variable **i**: auxiliary space $O(1)$. **binary_search** uses three index variables, **lo**, **hi**, and **mid**: auxiliary space $O(1)$ as well, since three variables is still a constant, regardless of how large n is. A *recursive* implementation of binary search, by contrast, would use $O(\log n)$ auxiliary space — not because the recursive version stores more data explicitly, but because each recursive call adds one more stack frame (Week 1’s stack-frame picture, reused here), and the recursion depth for binary search is itself $O(\log n)$, matching the number of passes

derived in Section 2.5. $O(1)$ auxiliary space is a property this course will price carefully later: Week 9 compares in-place sorting algorithms against mergesort precisely on this axis. The underlying intuition is Week 1's: each variable, wherever it lives — stack or heap — occupies one cell, and counting cells is exactly what $S(n)$ does.

2.7 Common Mistakes and What Comes Next

Three mistakes recur often enough with this material that they are worth naming explicitly, so that each can be caught in your own writing before it becomes a habit. **Treating $T(n) = O(n)$ as an exact count:** $O(n)$ names a whole *class* of functions that grow no faster than some constant multiple of n ; it is not a claim that the algorithm performs exactly n steps. **Stating a bound with no case:** “ $O(n)$ ” by itself, with no mention of best, worst, or average, is an unfalsifiable claim, because the same algorithm can have entirely different bounds in different cases, as Section 2.5 showed for linear search. **Mixing up O and Θ :** O is only an upper bound, possibly loose; only Θ asserts that the growth rate matches *exactly*, from both above and below. The habit that avoids all three mistakes at once: name the operation being counted, give the bound, state the case, and point to the specific input that exhibits it — exactly the pattern followed by every worked example in this chapter.

Next week turns to Arrays and the Stack ADT — the first data structure this course will analyse end to end, using precisely the cost-model vocabulary just built here: a contract (Week 1's ADT idea), an implementation, and now a cost, stated with its case.

Exercises

1. The calculator's symbol table doubles in size, from n to $2n$ names. For each of the following operations, state — and justify from its asymptotic class — roughly how much more expensive the operation becomes: (a) a linear scan of the table; (b) a binary search of the table (assume it is kept sorted); (c) the nested-loop duplicate check of Section 2.5.
2. Trace `linear_search` comparison by comparison on `A = [5, 15, 25, 35]` (so $n = 4$) searching for `key = 25`. State the number of comparisons performed, and say whether this run exhibits the best case, the worst case, or neither.
3. Show that $T(n) = 5n + 10$ is $O(n)$ by exhibiting valid constants c and n_0 and verifying the defining inequality holds for all $n \geq n_0$.
4. Show that $T(n) = 2n^2 + 3n$ is $\Theta(n^2)$ by exhibiting valid constants c_1 , c_2 , and n_0 for both the lower and the upper bound, and verifying each inequality.
5. Trace `binary_search` on the same array used in Section 2.5, `A = [3, 7, 12, 20, 30, 44, 50, 68, 81]` ($n = 9$), this time searching for `key = 20`. Produce a pass-by-pass table of `lo`, `hi`, `mid`, and the comparison result, and state how many passes the search takes.

Solutions

1. Doubling the table:

- (a) A linear scan's worst/average-case cost is $\Theta(n)$. Doubling n to $2n$ scales the cost by the ratio $T(2n)/T(n) = 2n/n = 2$: the linear scan roughly *doubles* in cost.

- (b) A binary search's worst-case cost is $\Theta(\log n)$. Doubling n changes the cost from $\log_2 n$ to $\log_2(2n) = \log_2 2 + \log_2 n = 1 + \log_2 n$: doubling the table costs exactly *one extra pass*, regardless of how large n already was.
- (c) The nested-loop duplicate check's cost is $\Theta(n^2)$. Doubling n changes the cost by the ratio $T(2n)/T(n) = (2n)^2/n^2 = 4n^2/n^2 = 4$: the duplicate check becomes *four times* as expensive.

These three ratios — doubles, one extra pass, four times — are exactly what the asymptotic classes $\Theta(n)$, $\Theta(\log n)$, and $\Theta(n^2)$ predict, and are the entire point of building the vocabulary this chapter introduces: it lets these predictions be made *before* either implementation is built or timed.

2. **Trace on A = [5, 15, 25, 35], key = 25:** compare A[0] = 5 against 25 ($5 \neq 25$, comparison 1); compare A[1] = 15 against 25 ($15 \neq 25$, comparison 2); compare A[2] = 25 against 25 ($25 = 25$, comparison 3, match found, return index 2). The search performs 3 comparisons. For $n = 4$ the best case would cost 1 comparison and the worst case would cost 4; this run costs 3, so it exhibits neither the best nor the worst case — it lands between the two, consistent with the average-case formula $\frac{n+1}{2} = \frac{5}{2} = 2.5$ from Section 2.3 (individual runs vary around that expected value; this one happens to cost slightly more than the average).
3. **$T(n) = 5n + 10$ is $O(n)$:** choose $c = 6$ and $n_0 = 10$. The inequality $5n + 10 \leq 6n$ simplifies (subtract $5n$ from both sides) to $10 \leq n$, which is exactly the condition $n \geq n_0 = 10$. Checking the boundary at $n = 10$: $5(10) + 10 = 60$, and $6(10) = 60$, so $60 \leq 60$ holds with equality. For any $n > 10$ the gap $6n - (5n + 10) = n - 10 > 0$ only grows, so the inequality continues to hold. Hence $T(n) = 5n + 10$ is $O(n)$ with $c = 6$, $n_0 = 10$. (Other valid choices exist too — e.g. $c = 15$, $n_0 = 1$ also works, since O only requires *some* valid c and n_0 to exist, not a unique pair.)
4. **$T(n) = 2n^2 + 3n$ is $\Theta(n^2)$:** **lower bound**, take $c_1 = 2$: the inequality $2n^2 + 3n \geq 2n^2$ simplifies (subtract $2n^2$ from both sides) to $3n \geq 0$, true for every $n \geq 0$, so this bound holds with $n_0 = 0$. **Upper bound**, take $c_2 = 5$: the inequality $2n^2 + 3n \leq 5n^2$ simplifies (subtract $2n^2$, then divide by 3) to $n \leq n^2$, which holds exactly when $n \geq 1$. Combining: for every $n \geq 1$, $2n^2 \leq 2n^2 + 3n \leq 5n^2$ holds simultaneously, which is the Θ sandwich with $c_1 = 2$, $c_2 = 5$, $n_0 = 1$. Hence $T(n) = 2n^2 + 3n$ is $\Theta(n^2)$.
5. **Binary search trace, key = 20, on A = [3, 7, 12, 20, 30, 44, 50, 68, 81] ($n = 9$):**

pass	lo	hi	mid	A[mid]
1	0	8	4	30 > 20
2	0	3	1	7 < 20
3	2	3	2	12 < 20
4	3	3	3	20 = 20 — found

Step by step: pass 1 starts at lo=0, hi=8, so mid = $0 + (8-0)/2 = 4$; A[4] = 30 > 20, so hi = mid - 1 = 3. Pass 2: lo=0, hi=3, so mid = $0 + (3-0)/2 = 1$; A[1] = 7 < 20, so lo = mid + 1 = 2. Pass 3: lo=2, hi=3, so mid = $2 + (3-2)/2 = 2$; A[2] = 12 < 20, so lo = mid + 1 = 3. Pass 4: lo=3, hi=3, so mid = $3 + (3-3)/2 = 3$; A[3] = 20 = 20, found, return mid = 3. The search takes 4 passes — one more than the 3-pass trace for key = 68 in Section 2.5, since $\lceil \log_2 9 \rceil = 4$ is the maximum number of passes binary search can take on this array, and this particular target happens to require that maximum.

Summary

- **Cost model:** count one well-chosen basic operation as a function of n , giving the exact cost function $T(n)$; compress that count into an asymptotic class; always state which case (best, worst, average) the count describes.
- $T(n)$ is the exact running-time cost function; $S(n)$ is the auxiliary space complexity — memory used beyond the input itself, as a function of n .

- **Asymptotic classes:** O (upper bound), Ω (lower bound), Θ (tight bound, both at once), with the strict forms o and ω available when the gap must be shown to be more than a constant factor.
- **Cases** — best, worst, average — are a property of the *input*; a bound is a property of the *function*. The two are orthogonal and must always be paired, never confused.
- **Our two searches:** linear search is $O(n)$ in the worst case ($\Theta(n)$ on average, $\Omega(1)$ best case); binary search (on a sorted array) is $O(\log n)$ in the worst case ($\Omega(1)$ best case); a nested-loop duplicate check is $\Theta(n^2)$. Both search algorithms use $O(1)$ auxiliary space; a recursive binary search would use $O(\log n)$ stack frames instead.
- Three habits avoid the recurring mistakes: never treat $O(n)$ as an exact step count, never state a bound without its case, and never substitute O for Θ when the growth rate is actually tight in both directions.

References

- [1] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Books, 2nd edition, 2008.
- [2] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.
- [3] Richard E. Neapolitan and Kumarss Naimipour. *Foundations of Algorithms*. Jones and Bartlett, 4th edition, 2010.
- [4] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.